

PilotFish Database Polling (SQL) Listener – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **scheduled database reader**. On a timer you configure, it connects to a database, runs a SQL-XML query (or a simple SQL statement you write in the console), and turns each result into an integration transaction for the rest of your route to process.

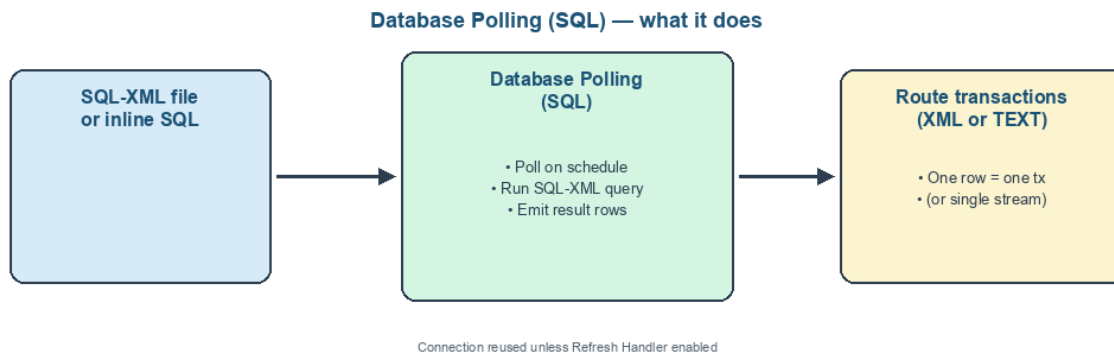


Figure 1: At a glance (plain English)

- **Polls** the database on a fixed interval (seconds).
- **Executes** SQL-XML from an input file or inline Write Query mode.
- **Starts work** in PilotFish for each result row (or one combined stream).
- **Reuses** the JDBC connection between polls unless you force a refresh.

Purpose

Deep dive into the **Database Polling (SQL)** listener in **eip.war.hs.26R1.11**: polling lifecycle, SQL-XML execution, result fan-out, connection handling, and every configuration option.

See also: [PilotFish-SQL-XML-Guide-26R1.11.pdf](#) — full SQL-XML language guide (commands, variables, result shape, XSLT patterns, demo examples).

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-db-1.0.1.jar</code>
Core JAR	<code>xcs-core-1.2.6.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.db.DatabaseSqlLi</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../Da</code>
Listener type string	Database Polling (SQL)
Description	Interprets SQL-XML as Database Statements
Help ID	<code>console.listener.databasepolling</code>
Categories	DATABASE, SQL

Related classes:

Class	Role
<code>DatabaseSqlListener</code>	Polling, <code>SQLXMLHandler</code> lifecycle, dispatch
<code>SQLXMLHandler</code>	JDBC connection, SQL-XML parse/execute
<code>CommonDatabaseUtils</code>	JDBC props, transaction isolation helpers
<code>TransactionBatch</code>	Batch-sensitive ordered response handling
<code>AbstractListener</code>	Base throttling, inactivity, scheduling

Derived from WAR bytecode (CFR). Narrative follows executable logic in `doListening` and `initializeHandler`.

1. High-level behavior

1. On **engage**, a single-thread scheduled executor runs `doListening()` at **Polling interval** (seconds). First execution is delayed **1 second**.
2. Each poll:
 - Initializes listener `TransactionData`
 - Validates required configuration
 - Optionally closes/rebuilds `SQLXMLHandler` when **Refresh Handler** is true
 - Loads query input (SQL-XML file **or** built-in simple SQL document)
 - Executes via `SQLXMLHandler.execute(data)`
 - Dispatches zero or more transactions from the XML output document

3. On **disengage**, the scheduler shuts down. On **system shutdown**, open JDBC connections are closed.

```
engageListener()
    scheduleAtFixedRate(doListening, 1s, PollingInterval)

doListening()
    initializeData() -> checkRequiredData()
    [Refresh Handler?] closeConnections()
    [Write Query?] build SQL-XML DOM from SQL + params
                    : load InputFile stream
    initializeHandler() -> SQLXMLHandler.execute()
    [Use Single Output Stream?] one notifyListeners
    else for each output element:
        XML output -> byte[] per element
        TEXT output -> concatenate tagged child text + attribute tags
        [Batch Sensitive?] attach TransactionBatch + index
        notifyListeners(...)
```

2. Configuration reference (module-specific)

2.1 Basic

Polling interval

Property	Value
UI label	Polling interval
Tag	PollingInterval
Type	Time (seconds)
Default	none (required)
Tooltip	How often the Input File query is executed.
Required	Yes

Invalid interval logs error and calls `notifyListenersOnError`; listener does not schedule.

Input file

Property	Value
UI label	Input file
Tag	InputFile
Type	File
Default	""
Tooltip	The SQL-XML query to be executed; required.
Visible when	Write Query = false

Resolved via `ConfigurationManager.getFile(..., loader)` and opened as `FileInputStream` on the listener transaction data before execute.

Write Query / SQL / Query Params

Label	Tag	Default	Notes
Write Query	<code>WriteQuery</code>	false	Switches from file to inline SQL
SQL	<code>Query</code>	""	Simple SQL text; required when Write Query true
Query Params	<code>QueryParams</code>	""	Table of positional parameters

When **Write Query** is enabled, `CommonDatabaseUtils.buildSimpleSqlXmlQuery(sql, params)` builds a DOM, serializes to bytes, and sets that as the transaction input stream. **Input file** is hidden.

2.2 Connection tab

Label	Tag	Default	Tooltip summary
Type	<code>UseDataSource</code>	false (JDBC Connection)	Data source vs direct JDBC
Data Source	<code>DataSource</code>	""	Container data source name
User name	<code>UserName</code>	""	DB username
Password	<code>Password</code>	""	DB password
JDBC driver	<code>JdbcDriver</code>	""	Driver class
JDBC URL	<code>JdbcURL</code>	""	Connection URL
Test Connection	(button)	—	Action <code>TestSqlConnection</code>

Dependencies: Type = Data Source shows **Data Source**; Type = JDBC Connection shows username, password, driver, URL.

2.3 Transaction Isolation tab

Label	Tag	Default
Transaction Isolation	<code>TransactionIsolationLevel</code>	level choice (None/unset driver default)
Driver Specific Level	<code>TransactionIsolationDriverSpecific</code>	

Driver Specific Level visible when isolation choice is **Driver Specific**. Values from `CommonDatabaseUtils.TRANSACTION_ISOLATION_LEVEL_CHOICES`.

2.4 Advanced

Label	Tag	Default	Nuances
Is Batch Sensitive?	BatchSensitive	false	Attaches TransactionBatch ; stops sequential batch on failure
Enable Timeout for Queries	EnableTimeout	false	Gates Timeout field
Timeout for Queries	Timeout	600	Seconds; passed to SQLXMLHandler.setTimeout
Disable Metadata	DisableMetadata	false	Skips metadata load in handler
Restrict Metadata To Catalog	RestrictMetadataToCatalog		JDBC metadata filter
Restrict Metadata To Schema	RestrictMetadataToSchema		Schema pattern
Restrict Metadata To Table(s)	RestrictMetadataToTable		Table pattern; default %
Use Single Output Stream	UseSingleOutputStream		One transaction for entire output if any child elements exist
Refresh Handler	RefreshHandler	false	Closes JDBC handler each poll before re-init
Error on Unknown Elements	ErrorOnUnknownElement		Strict SQL-XML element handling
Use JDBC Identity for Inserts	UseJdbcGeneratedKeys		Overrides SQL-XML IdentityQuery when true

2.5 JDBC Properties tab

Label	Tag	Purpose
JDBC Properties	JDBCProperties	Name/value pairs applied to JDBC Properties

2.6 Debug tab

Label	Tag	Default
Log Metadata	LogMetadata	false
SQLXML Logging	LogSQL	false

2.7 Scheduling tab

Inherited scheduler descriptor: start/end schedule, exclude days/dates, time zone.

2.8 Runtime-read options (not in listener descriptor)

`initializeHandler` also reads **KeepConnection** and **UseJdbcTyping** (defaults false if absent). These appear on other SQL-XML modules (e.g. Database SQL transport/processor) but are **not** declared in `DatabaseSqlListener.getConfigurationDescriptor()`. They still affect connection reuse and type mapping when present in route XML.

3. Dispatch algorithms

3.1 Use Single Output Stream

When true, after execute the listener checks whether the output DOM root has any **Element** children. If yes, it calls `notifyListeners` once with bytes read from `data.getDataStream()` (post-execute stream) and **returns** — no per-row split.

3.2 Per-element fan-out (default)

Iterates child nodes of the output document element. Skips non-element nodes.

Output type (<code>ec.getXMLOutOutputType()</code>)	Behavior
XML	Transform each element to bytes; notifyListeners per element
TEXT	Concatenate text from tags in <code>getXMLOutOutputTags()</code> ; copy attribute-tag children into TransactionAttributes

3.3 Batch Sensitive

When true:

- Creates `TransactionBatch("com.pilotfish.DatabaseSqlListener")`
- Each element gets attributes:
 - `com.pilotfish.eip.util.TransactionBatch.instance` → batch object
 - `com.pilotfish.DatabaseSqlListener` → `MutableInt` index
- `batch.incrementBatchSize()` per element

Downstream modules use the batch to enforce ordered response execution and stop on failure.

4. Transaction attributes

The listener does not register design-time attribute metadata. Runtime attributes include:

Key	When set
Tags from SQL-XML TEXT output	Names in <code>ec.getXMLOutAttributeTags()</code>
<code>com.pilotfish.eip.util.TransactionBatchSensitiveInstance</code>	Batch sensitive instance
<code>com.pilotfish.DatabaseSqlListener</code>	Batch index (MutableInt)
Inherited listener attributes	Via <code>loadTransactionAttributes</code> on dispatch

SQL-XML query results themselves become the transaction **data stream** (XML or TEXT bytes).

5. Error handling

Condition	Behavior
Any exception in <code>doListening</code>	<code>closeConnections()</code> then <code>notifyListenersOnError</code>
Invalid polling interval	Error on engage; no scheduler
Parser failure (Write Query)	<code>EIPEXception("Could not load simple SQL")</code>
Handler execute failure	Connection closed; listener error notification

6. Worked example (Batch Sensitive + multi-row)

Configuration: Batch Sensitive = true, Use Single Output Stream = false, query returns 3 XML row elements.

1. Poll runs; handler executes SELECT returning 3 rows as 3 XML child elements.
2. Listener emits **3 transactions**, each tagged with batch instance and index 0, 1, 2.
3. If the route's response processor fails on index 1, batch semantics stop further responses for that batch (per `TransactionBatch` contract).

With **Use Single Output Stream** = true and the same query, **one** transaction carries the combined stream regardless of row count.

7. Known quirks

1. **Refresh Handler vs KeepConnection:** Refresh closes handler each poll; KeepConnection (runtime tag) keeps JDBC open between polls when handler is not refreshed.
2. **Single output mode** checks for element children only — whitespace/text nodes do not trigger dispatch.
3. **Write Query** builds SQL-XML in memory; large parameter lists still serialize through DOM.
4. **Use JDBC Identity** defaults **true** and overrides custom identity queries in SQL-XML.

5. **Metadata table default %** can still be expensive on large catalogs — tighten catalog/schema/table patterns.
6. **Immediate save** is set on the configuration descriptor (`setImmediateSave(true)`).
7. **KeepConnection / UseJdbcTyping** affect behavior but are not exposed in this listener's UI descriptor.

8. Operational recommendations

Goal	Suggested settings
Stable JDBC under firewalls	Enable query timeout; test connection from console
High row volume	Tune polling interval; use throttling (base listener)
Ordered DB callbacks	Batch Sensitive = true + matching response handling
One document per poll	Use Single Output Stream
Connection leaks / stale pools	Refresh Handler = true, or KeepConnection = false
Faster metadata	Restrict Metadata To Schema/Table; Disable Metadata if safe

9. Inherited AbstractListener options

From `AbstractListener.getBaseConfiguration()` (Advanced, Throttling, Inactivity tabs):

Label	Tag	Default
Initialize on trigger only	<code>IS_TRIGGERABLE_LISTENER</code>	<code>false</code>
Allow command-line invocation	<code>CLIAllowed</code>	<code>false</code>
Restart on listening error	<code>RESTART_ON_ERROR_TAG</code>	<code>false</code>
FIFO Queue Name	<code>FIFOQueueName</code>	<code>""</code>
FIFO Queue Delay	<code>FIFOQueueDelay</code>	<code>500</code>
Throttling Mode	<code>ThrottlingMode</code>	<code>None</code>
Throttling Mechanism	<code>ThrottlingMechanism</code>	<code>Blocking</code>
Max Concurrent Messages	<code>ThrottlingConcurrentMessages</code>	
Timed Emission Interval	<code>ThrottlingTimedInterval</code>	<code>1</code>
Synchronous Timeout Interval	<code>ThrottlingSynchronousTimeout</code>	<code>500</code>
Enable Inactivity Monitor	<code>CheckInactivity</code>	<code>false</code>
Min. Transactions to Expect	<code>InactivityTransactionCount</code>	
Monitoring Interval	<code>InactivityPolling</code>	<code>500</code>
Times to Monitor	<code>MonitoringTime</code>	builder default
Days to Exclude from Monitoring	<code>InactivityExcludedDays</code>	<code>''</code>

Label	Tag	Default
Include Errors in Transaction Count	InactivityIncludeErrors	false

Scheduling: SchedulerStartTag, SchedulerEndTag, ExcludeDaysTag, ExcludeDatesTag, TimeZone.

10. Source index

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
DatabaseSqlListener	com/pilotfish/eip/modules/db/DatabaseSqlListener.java
SQLXMLHandler	com/pilotfish/eip/util/SQLXMLHandler.java
CommonDatabaseUtils	com/pilotfish/eip/util/CommonDatabaseUtils.java
AbstractListener	com/pilotfish/eip/extend/AbstractListener.java

Deep-dive documentation from WAR decompile — 26R1.11 — August 2026.